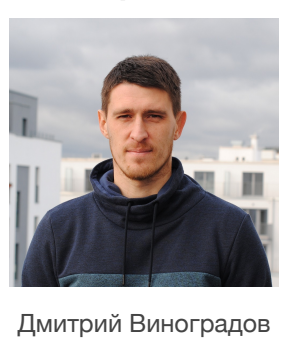
Language

Автор сайта



Дмитрий Виноградов

Подробнее можно посмотреть или почитать.

Никакие другие люди не имеют к этому сайту никакого отношения и просто занимаются плагиатом.

Kotlin
Корутины
- suspend function
- scope
- builders
- exceptions
- flow

Социальные сети

В канале я публикую ссылки на интересные и полезные статьи по Android

В чате можно обсудить вопросы и проблемы, возникающие при разработке

Группа ВКонтакте

StartAndroid
15 086 подписчиков
Подписаться

Поддержка проекта

Яндекс
410011180491924

WebMoney
R248743991365
Z551306702056

Paypal

Урок 27. Drawing

18 июля 2023

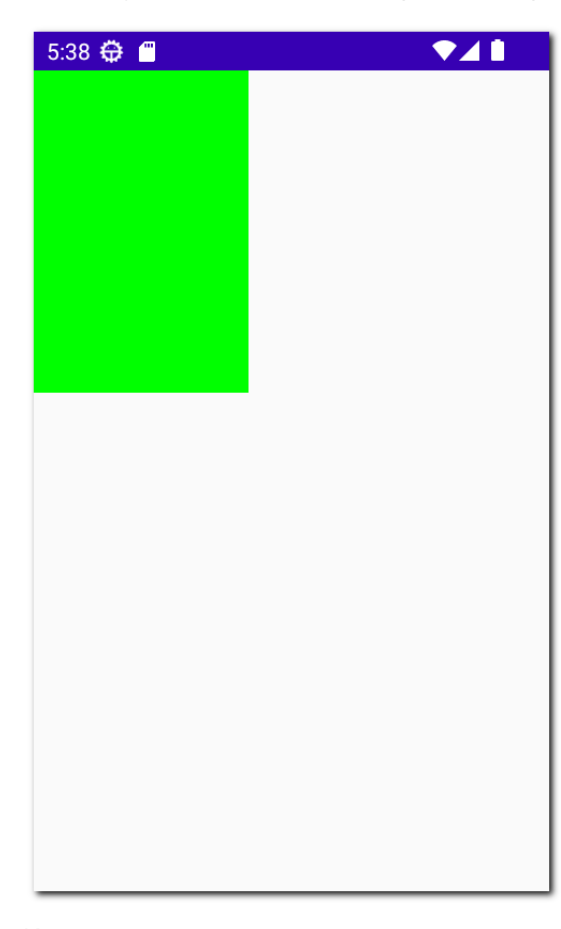
В этом уроке рассматриваем инструменты для рисования в Compose

Если вы рисовали в Android до Compose, то помните, что там есть Canvas (канва). Это холст, на котором мы рисуем всевозможные линии, круги, квадраты и прочие графические примитивы.

В Compose есть функция Canvas с похожим функционалом:

```
Canvas(modifier = Modifier.fillMaxSize()) {
    drawRect(
        color = Color.Green,
        size  = Size(200f, 300f)
    )
}
```

Используем drawRect, чтобы нарисовать прямоугольник.



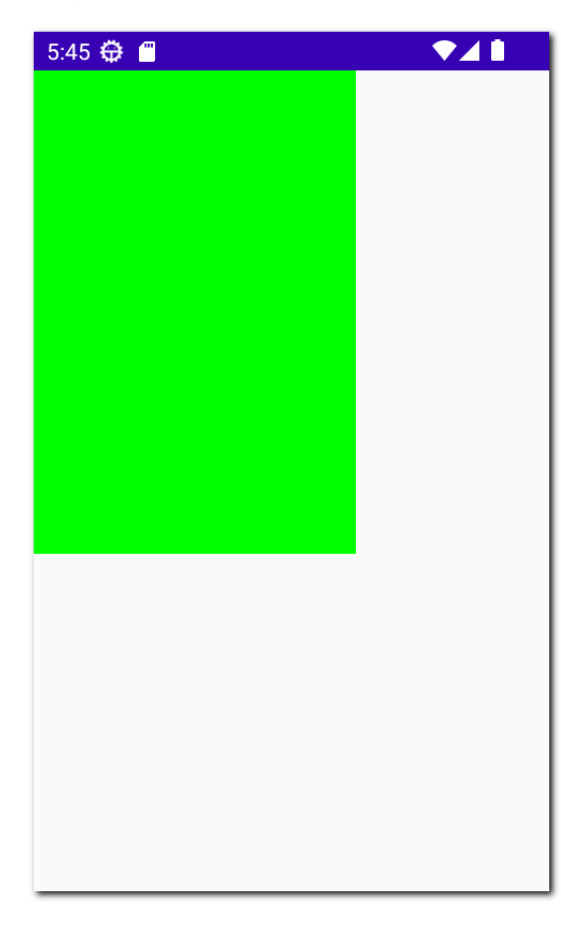
Кроме прямоугольника нам доступны следующие фигуры: drawRoundedRect(), drawLine(), drawCircle(), drawOval(), drawArc(), drawPoints(), drawPath(). Не будем рассматривать их подробно, остановимся только на общих и интересных моментах.

Вместо обычного цвета мы можем использовать кисть, которая дает возможность рисовать градиентом или картинкой. Подробнее об этом можно почитать в официальном доке и в статье.

Координаты задаем в пикселах. Если у вас размеры DP, то используйте расширение Dp.toPx, чтобы конвертировать в пикселы:

```
Canvas(modifier = Modifier.fillMaxSize()) {
    drawRect(
        color = Color.Green,
        size = Size(200.dp.toPx(), 300.dp.toPx())
    )
}
```

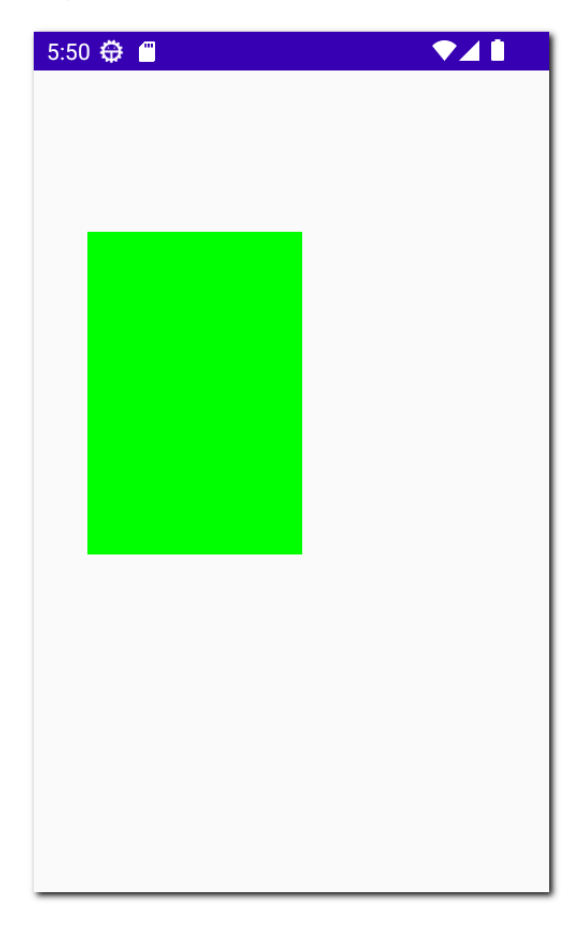
Результат:



Чтобы нарисовать фигуру в произвольном месте экрана, используем объект Offset, в котором указываем размер сдвига от верхнего левого угла:

```
Canvas(modifier = Modifier.fillMaxSize()) {
    drawRect(
        color = Color.Green,
        size = Size(200f, 300f),
        topLeft = Offset(50f, 150f)
    )
}
```

Результат:



Transformations

Трансформации могут быть полезными, когда наш рисунок состоит из нескольких примитивов и нам нужно изменить размер, повернуть или сдвинуть его. Рассмотрим их подробнее, потому что там есть некоторые сложности.

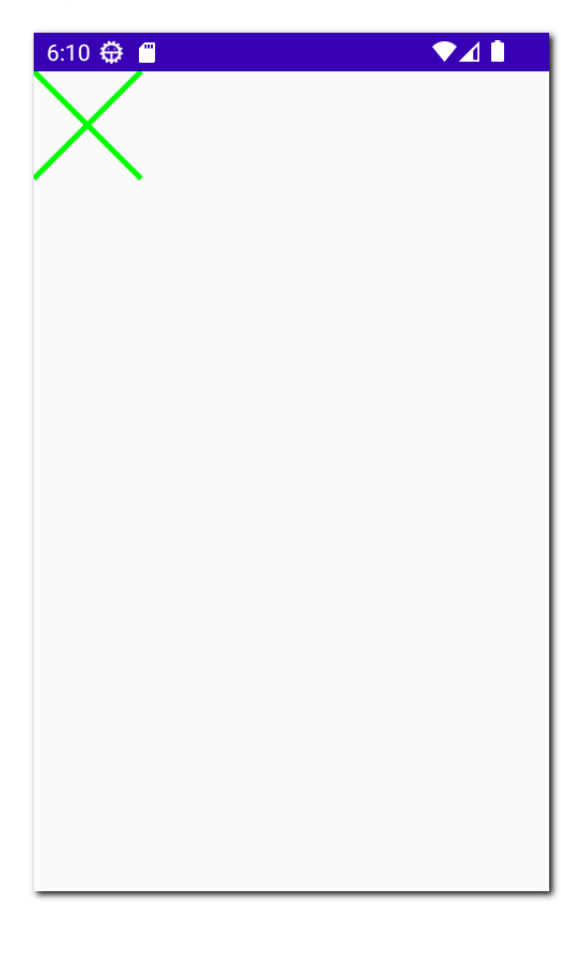
scale

scale меняет размеры того, что в нем нарисовано.

В качестве примера рассмотрим рисование крестика:

```
Canvas(modifier = Modifier.fillMaxSize()) {
    drawLine(
        color = Color.Green,
        start = Offset(0f,0f),
        end = Offset(100f, 100f),
        strokeWidth = 5f
    )
    drawLine(
        color = Color.Green,
        start = Offset(100f,0f),
        end = Offset(0f, 100f),
        strokeWidth = 5f
    )
}
```

Результат:



Чтобы мне не повторять этот код во всех примерах, я вынесу его в отдельную функцию:

```
private fun DrawScope.cross() {
    drawLine(
        color = Color.Green,
        start = Offset(0f,0f),
        end = Offset(100f, 100f),
        strokeWidth = 5f
    )
    drawLine(
        color = Color.Green,
        start = Offset(100f,0f),
        end = Offset(0f, 100f),
        strokeWidth = 5f
    )
}
```

Теперь пример выглядит так:

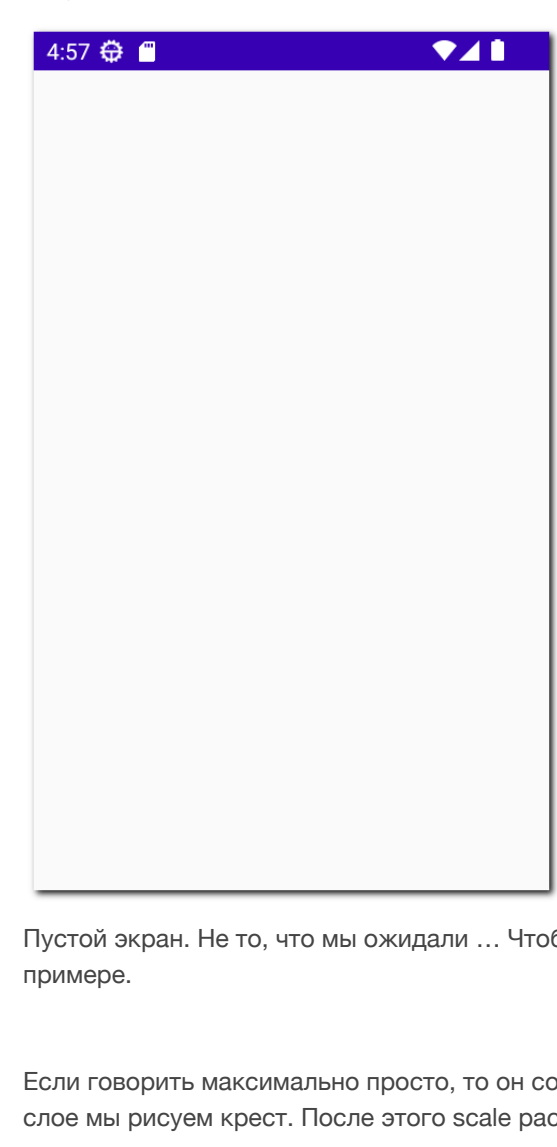
```
Canvas(modifier = Modifier.fillMaxSize()) {
    cross()
}
```

Попробуем увеличить крест в два раза с помощью scale:

```
Canvas(modifier = Modifier.fillMaxSize()) {
    scale(scaleX = 2f, scaleY = 2f) {
        cross()
    }
}
```

Мы вызываем scale, передаем ему, что надо увеличить размер в два раза по оси X и оси Y, и внутри scale вызываем cross.

Результат:

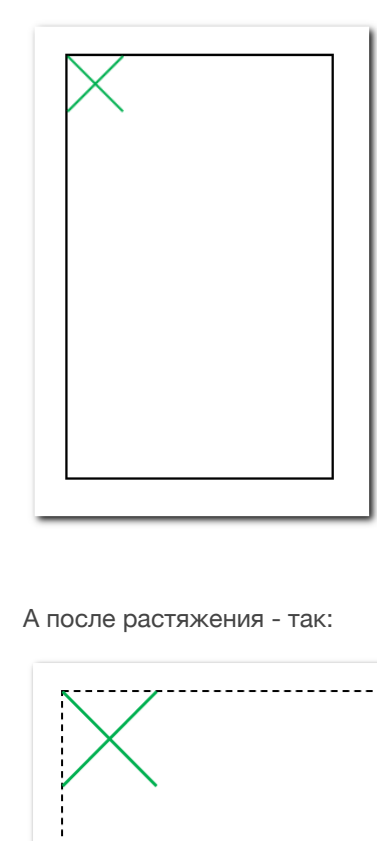


Пустой экран. Не то, что мы ожидали ... Чтобы понять почему так получилось, надо понимать, что делает scale в нашем примере.

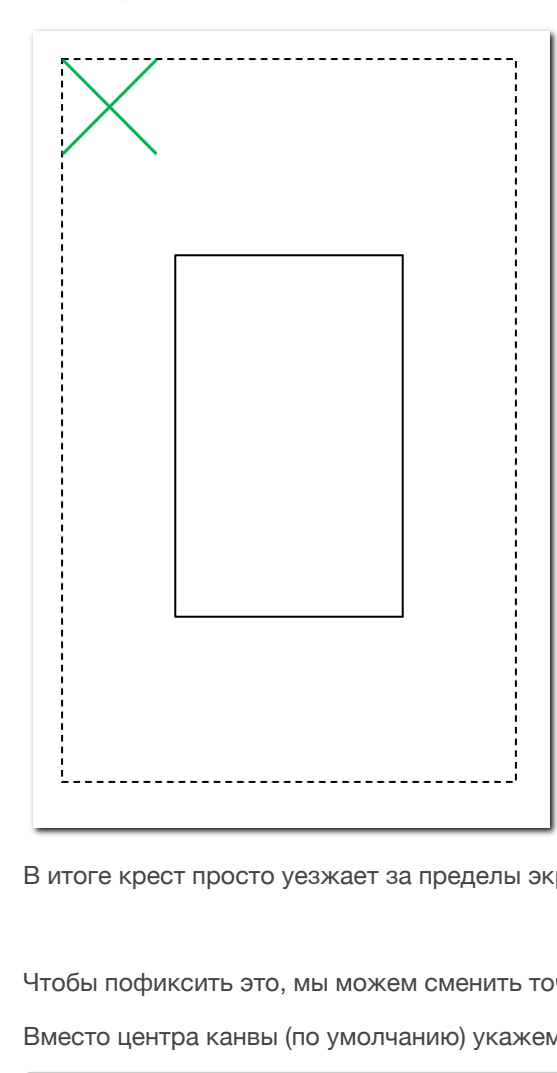
Если говорить максимально просто, то он создает на канве временный слой, размер которого равен размеру канвы. На этом слое мы рисуем крест. После этого scale растягивает этот слой в два раза по горизонтали и вертикали и переносит его содержимое на канву.

Проблема в том, что растягивание слоя идет относительно центра канвы.

Т.е. до растяжения слой с крестом выглядит так:



А после растяжения - так:



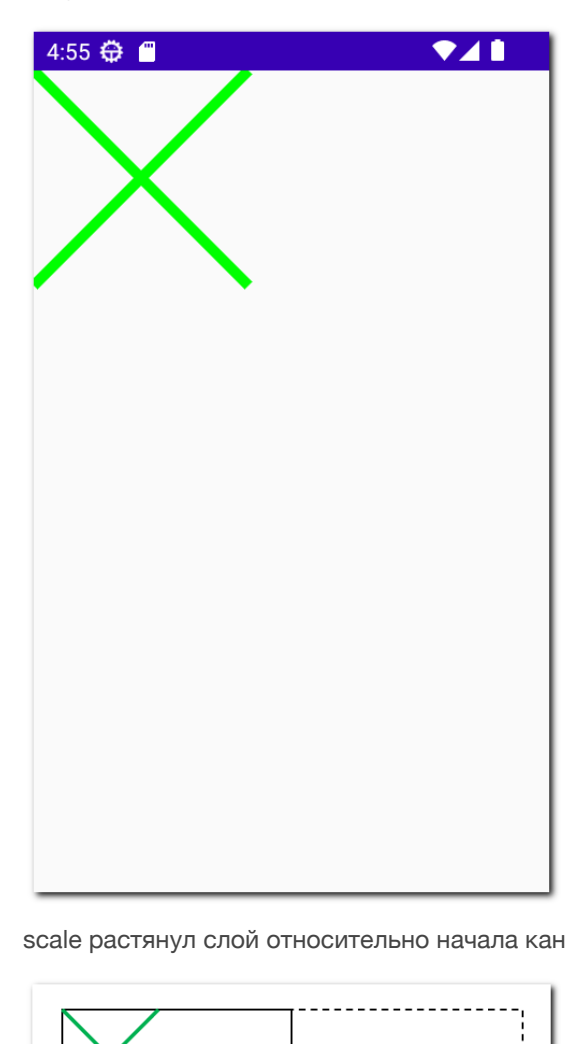
В итоге крест просто уезжает за пределы экрана.

Чтобы пофиксить это, мы можем сменить точку, относительно которой scale выполняет растяжение.

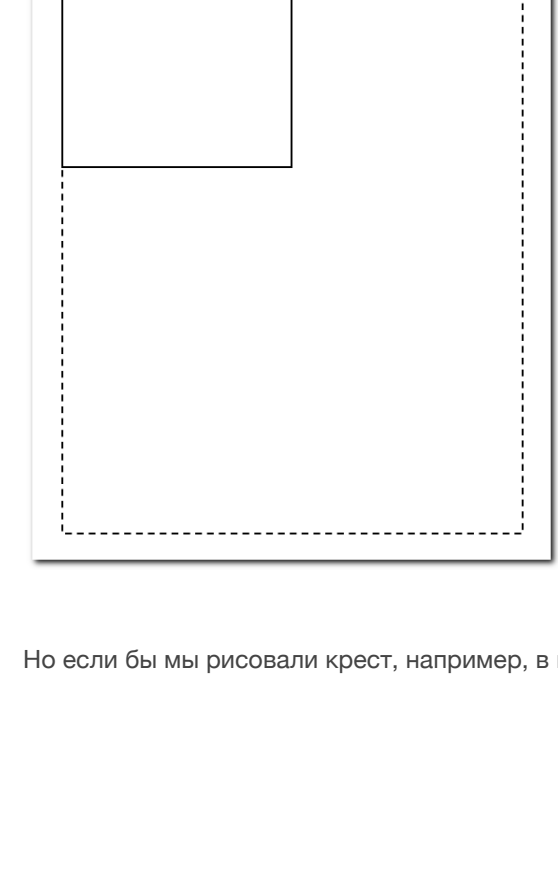
Вместо центра канвы (по умолчанию) укажем верхний левый угол, т.е. (0,0):

```
Canvas(modifier = Modifier.fillMaxSize()) {
    scale(scaleX = 2f, scaleY = 2f, pivot = Offset(0f,0f)) {
        cross()
    }
}
```

Результат:



scale растянул слой относительно начала канвы и крест остался на экране



Но если бы мы рисовали крест, например, в правом нижнем углу канвы, то он бы уехал за пределы экрана.

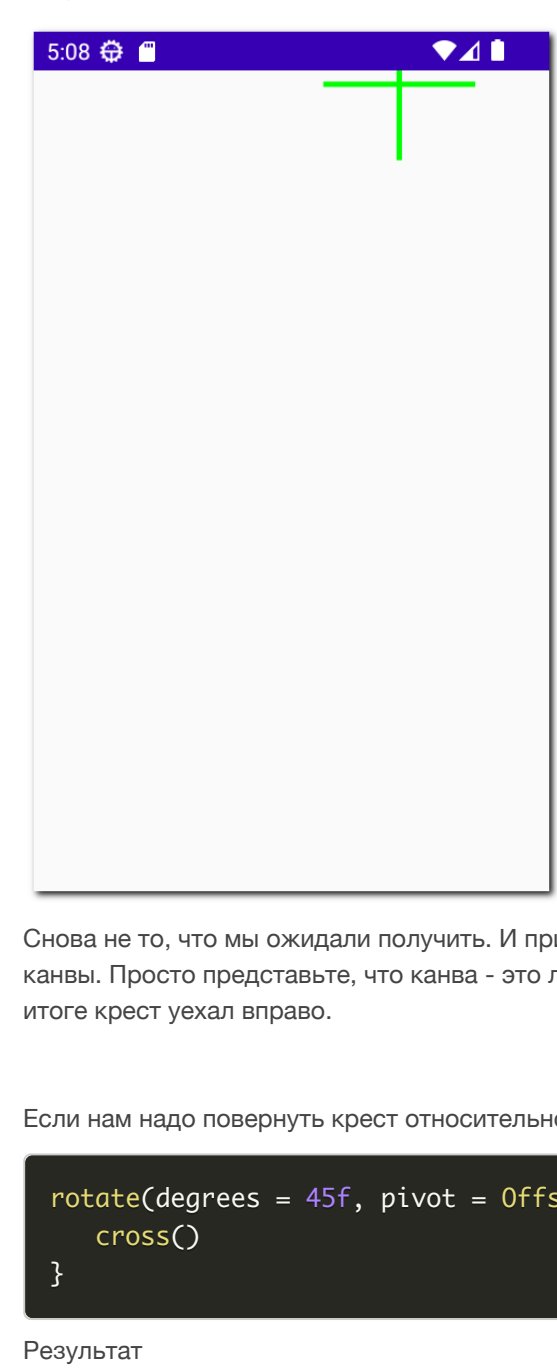
rotate

Следующая трансформация - поворот.

Повернем крест на 45 градусов:

```
rotate(degrees = 45f) {
    cross()
}
```

Результат:

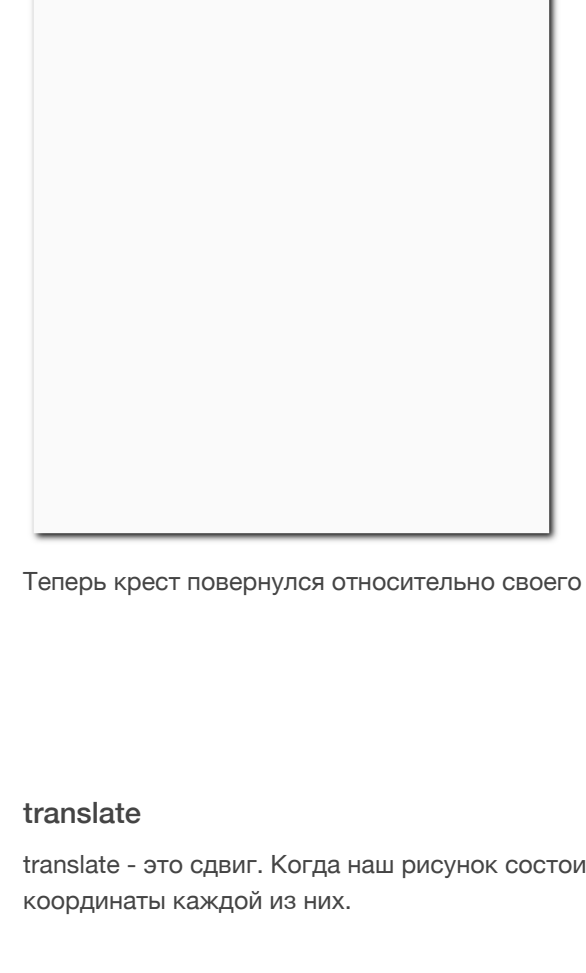


Снова не то, что мы ожидали получить. И причина та же - точка, относительно которой работает поворот. Это снова центр канвы. Просто представьте, что канва - это лист бумаги. В центр листа мы втыкаем иголку и крутим лист на 45 градусов. В итоге крест уехал вправо.

Если нам надо повернуть крест относительно его же центра, то можно указать, что точка поворота - это центр креста.

```
rotate(degrees = 45f, pivot = Offset(50f,50f)) {
    cross()
}
```

Результат



Теперь крест повернулся относительно своего центра.

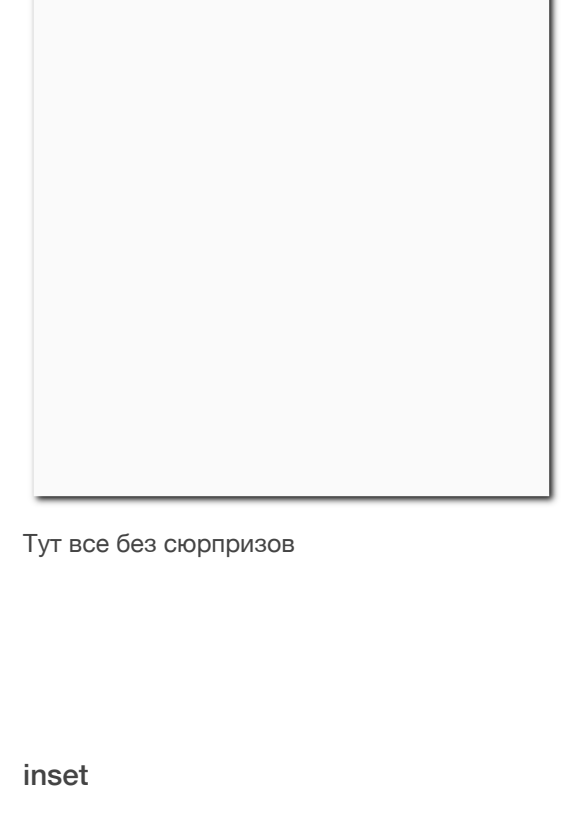
translate

translate - это сдвиг. Когда наш рисунок состоит из нескольких фигур гораздо проще сдвинуть их все сразу, чем менять координаты каждой из них.

Сдвинем крест:

```
translate(left = 100f, top = 100f) {
    cross()
}
```

Результат



Тут все без сюрпризов

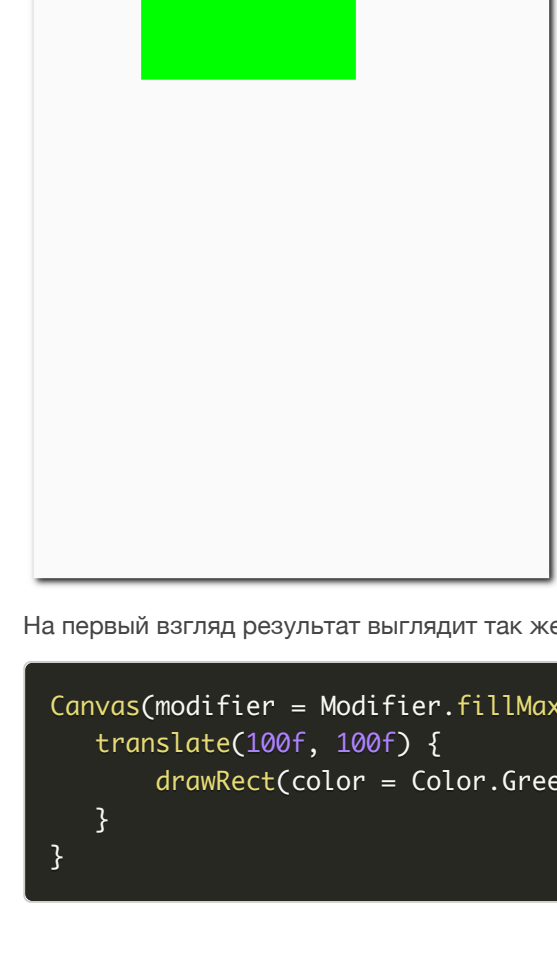
inset

Эта трансформация урезает размеры канвы. Т.е. это что-то типа padding.

Используем ее с квадратом:

```
Canvas(modifier = Modifier.fillMaxSize()) {
    inset(100f, 100f) {
        drawRect(color = Color.Green, size = Size(200f,200f))
    }
}
```

Результат



На первый взгляд результат выглядит так же, как если бы мы просто сдвинули квадрат с помощью translate:

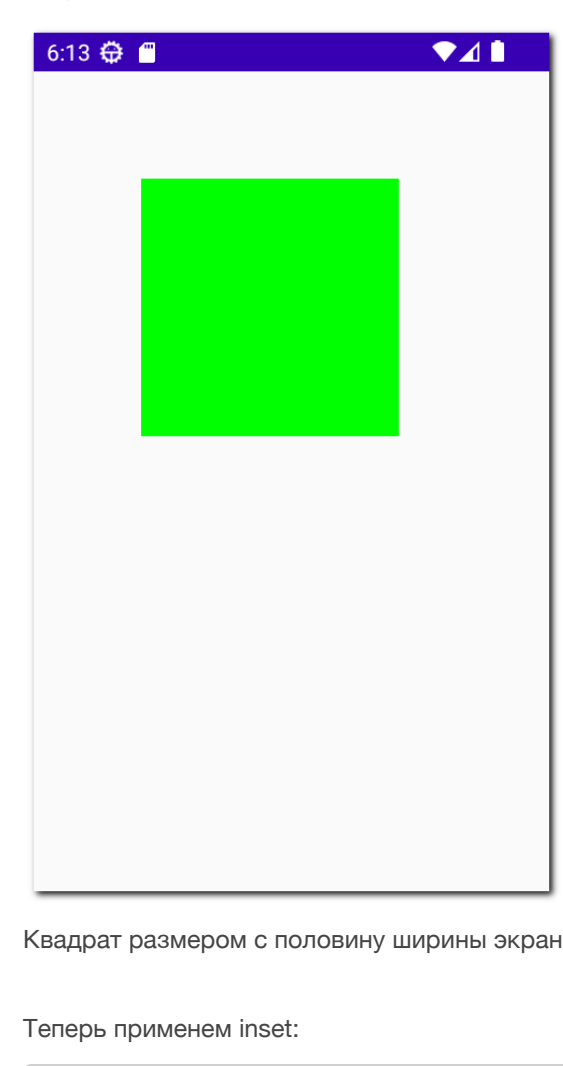
```
Canvas(modifier = Modifier.fillMaxSize()) {
    translate(100f, 100f) {
        drawRect(color = Color.Green, size = Size(200f,200f))
    }
}
```

Но если размер нашей фигуры зависит от размеры канвы, то разница станет заметна. Поставим квадрату размер в половину ширины канвы. Чтобы получить размер канвы, используем size, который доступен нам в DrawScope

Выполним сначала translate:

```
Canvas(modifier = Modifier.fillMaxSize()) {
    Log.d(TAG, "size = $size")
    translate(100f, 100f) {
        drawRect(color = Color.Green, size = Size(size.width/2, size.width/2))
    }
}
```

Результат

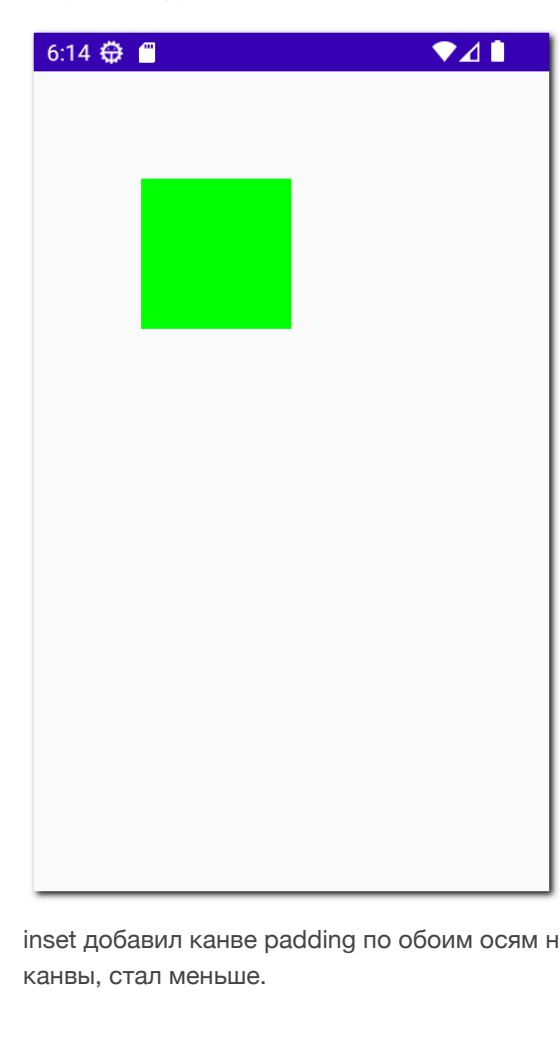


Квадрат размером с половину ширины экрана сдвинут на (100, 100). Все верно.

Теперь применим inset:

```
Canvas(modifier = Modifier.fillMaxSize()) {
    inset(100f, 100f) {
        drawRect(color = Color.Green, size = Size(size.width/2, size.width/2))
    }
}
```

Результат другой:



inset добавил канве padding по обоим осям на 100. Канва стала меньше. А значит и квадрат, который зависит от размера канвы, стал меньше.

Комбинирование

Мы можем использовать сразу несколько трансформаций. И это дает нам возможность немного упростить работу с pivot точками.

Трансформации scale и rotate по умолчанию работают относительно центра канвы. Чтобы они адекватно работали с нашим крестом, мы смещали их pivot точки.

Но можно сделать и обратное действие. Перепишем код креста так, чтобы его центр находился в центре канвы. Тогда scale и rotate будут для него работать так, как надо и без указания всяких точек. А после этого мы просто используем translate, чтобы поместить крест в нужную нам позицию

Рисуем крест в центре канвы:

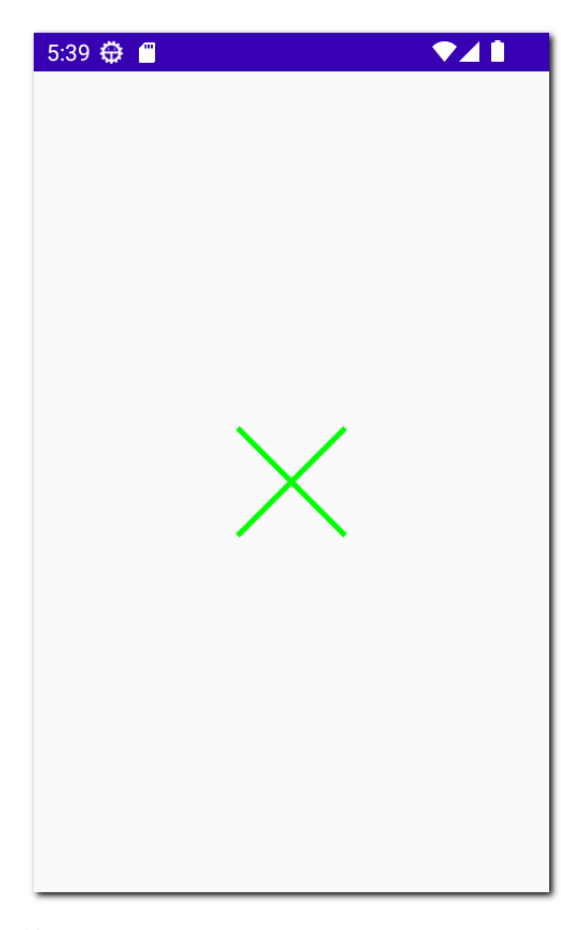
```
private fun DrawScope.cross() {
    drawLine(
        color = Color.Green,
        start = center.plus(Offset(-50f,-50f)),
        end = center.plus(Offset(50f,50f)),
        strokeWidth = 5f
    )
    drawLine(
        color = Color.Green,
        start = center.plus(Offset(50f,-50f)),
        end = center.plus(Offset(-50f,50f)),
        strokeWidth = 5f
    )
}
```

center - это точка центра канвы, доступная нам в DrawScope. К ней мы просто добавляем отступы с разных сторон, чтобы получить стартовые и конечные точки для линий креста.

Выведем его на экран без трансформаций:

```
Canvas(modifier = Modifier.fillMaxSize()) {
     cross()
}
```

Результат:



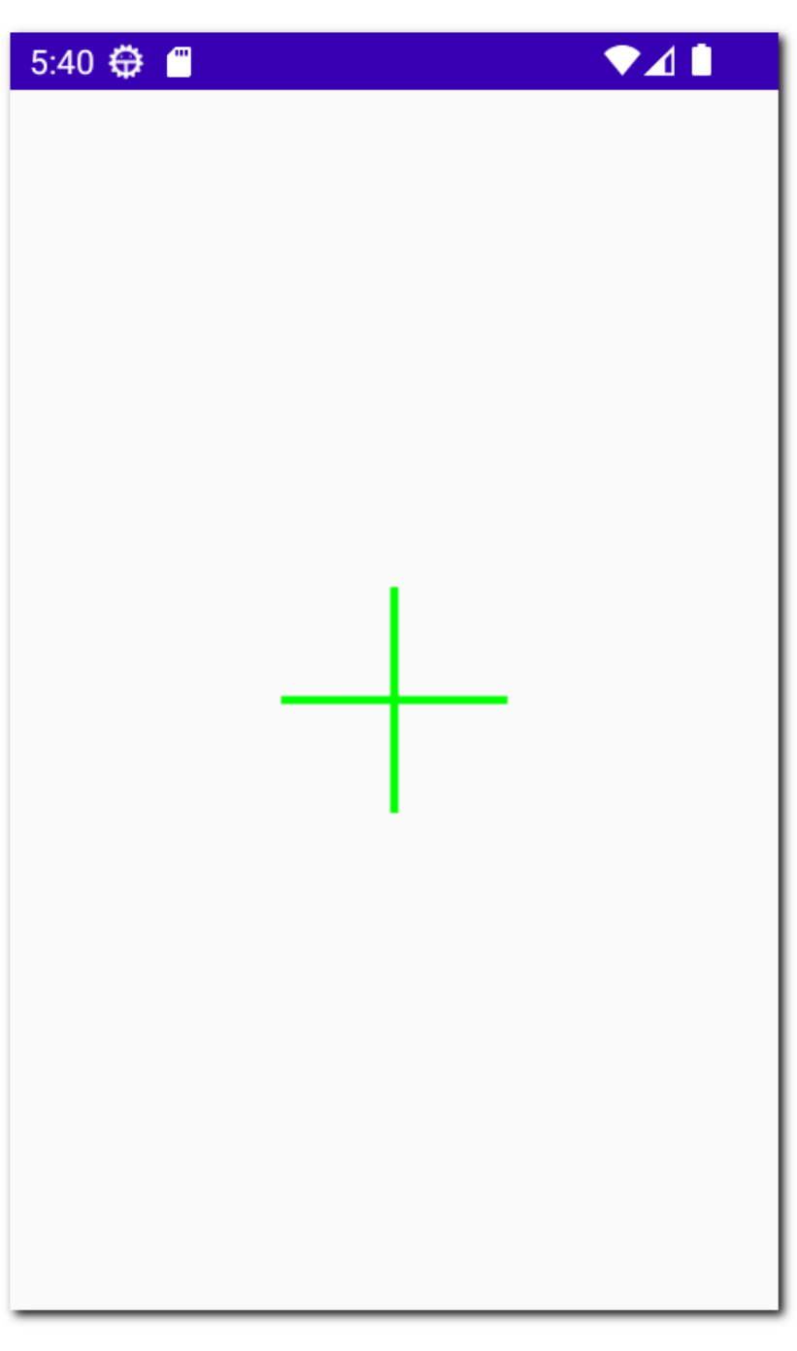
Крест в центре

Повернем его с помощью rotate:

```
Canvas(modifier = Modifier.fillMaxSize()) {
    rotate(degrees = 45f) {
        cross()
    }
}
```

Теперь нам не надо указывать pivot точку для rotate. Вариант по умолчанию (центр) нам подходит.

Результат:

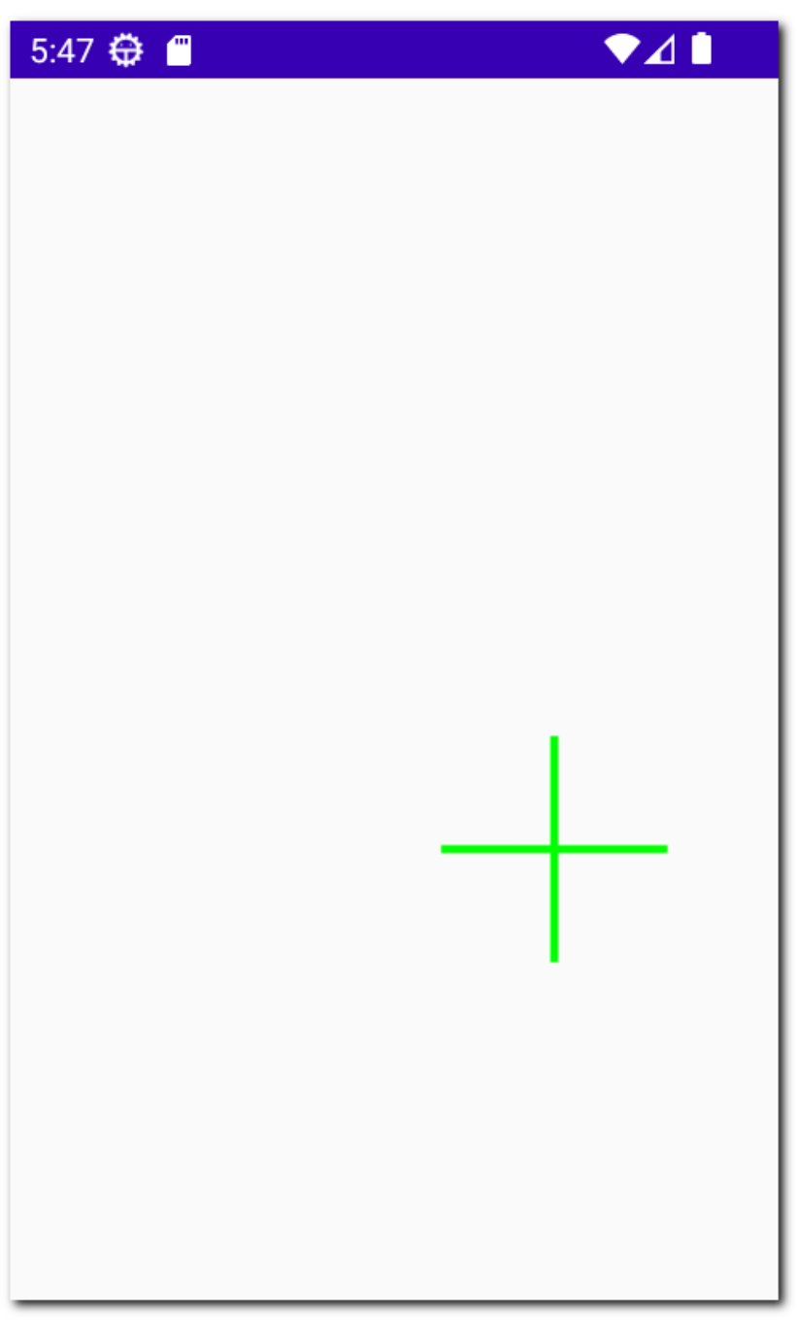


Крест повернулся, как и ожидалось.

Если крест нужен нам не в центре, то мы можем его подвинуть на нужную нам позицию, добавив translate:

```
Canvas(modifier = Modifier.fillMaxSize()) {
    translate(100f, 100f) {
        rotate(degrees = 45f) {
            cross()
        }
    }
}
```

Результат:



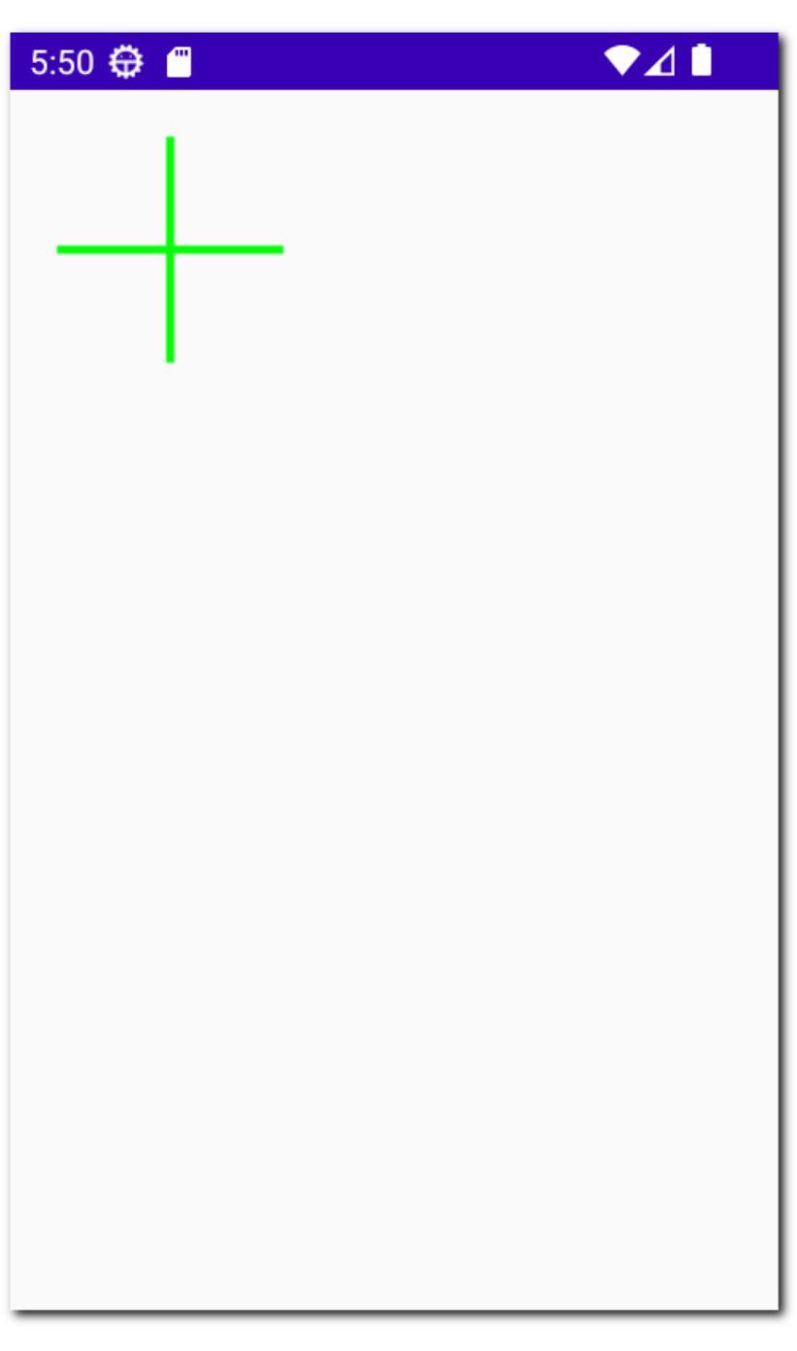
Крест сдвинулся на (100,100) от центра, где он находился.

Если нам надо не просто подвинуть его относительно центра, а передвинуть в конкретную точку, то это можно сделать, например, так:

```
Canvas(modifier = Modifier.fillMaxSize()) {
    translate( - center.x + 100f, - center.y + 100f) {
        rotate(degrees = 45f) {
            cross()
        }
    }
}
```

В translate мы сначала смещаем координаты в точку (0,0), а уже оттуда в точку (100,100).

Результат:



Для комбинирования трансформаций есть специальный метод - withTransform.

```
Canvas(modifier = Modifier.fillMaxSize()) {
    withTransform({
        translate( - center.x + 100f, - center.y + 100f)
        rotate(degrees = 45f)
    }) {
        cross()
    }
}
```

Рекомендуется использовать его вместо того, чтобы вкладывать одну трансформацию в другую

Text

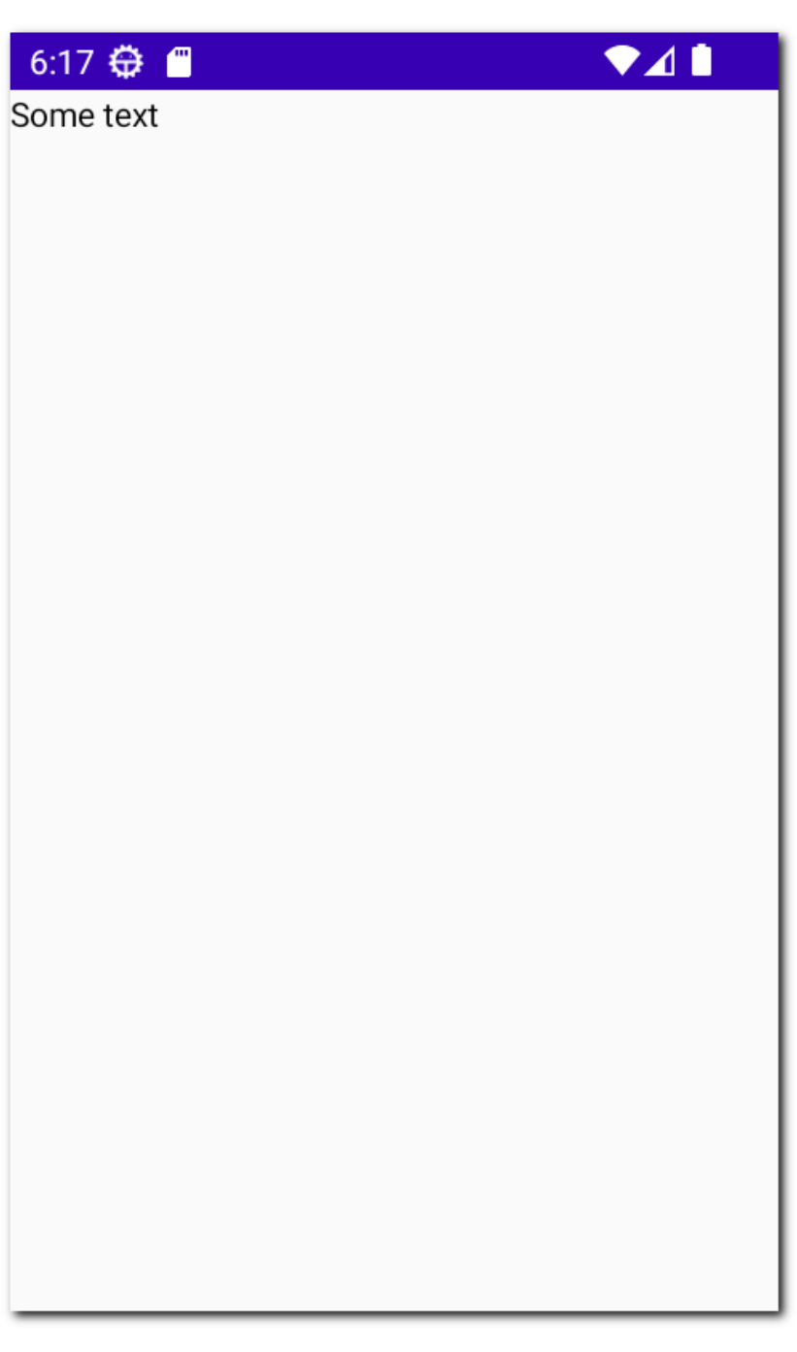
Канва дает возможность рисовать текст, но для этого нужен TextMeasurer.

```
val textMeasurer = rememberTextMeasurer()
Canvas(modifier = Modifier.fillMaxSize()) {
    val measuredText = textMeasurer.measure(
        text = AnnotatedString("Some text")
    )
    drawText(measuredText)
}
```

С помощью rememberTextMeasurer мы создаем TextMeasurer. Затем вызываем его метод measure, чтобы он замерил, как будет нарисован текст. В measuredText мы получаем результаты этих замеров.

Например, нам доступен будущий размер текста - measuredText.size, или количество строк в нем - measuredText.lineCount.

Объект measuredText мы передаем в drawText, который выведет на экран текст:



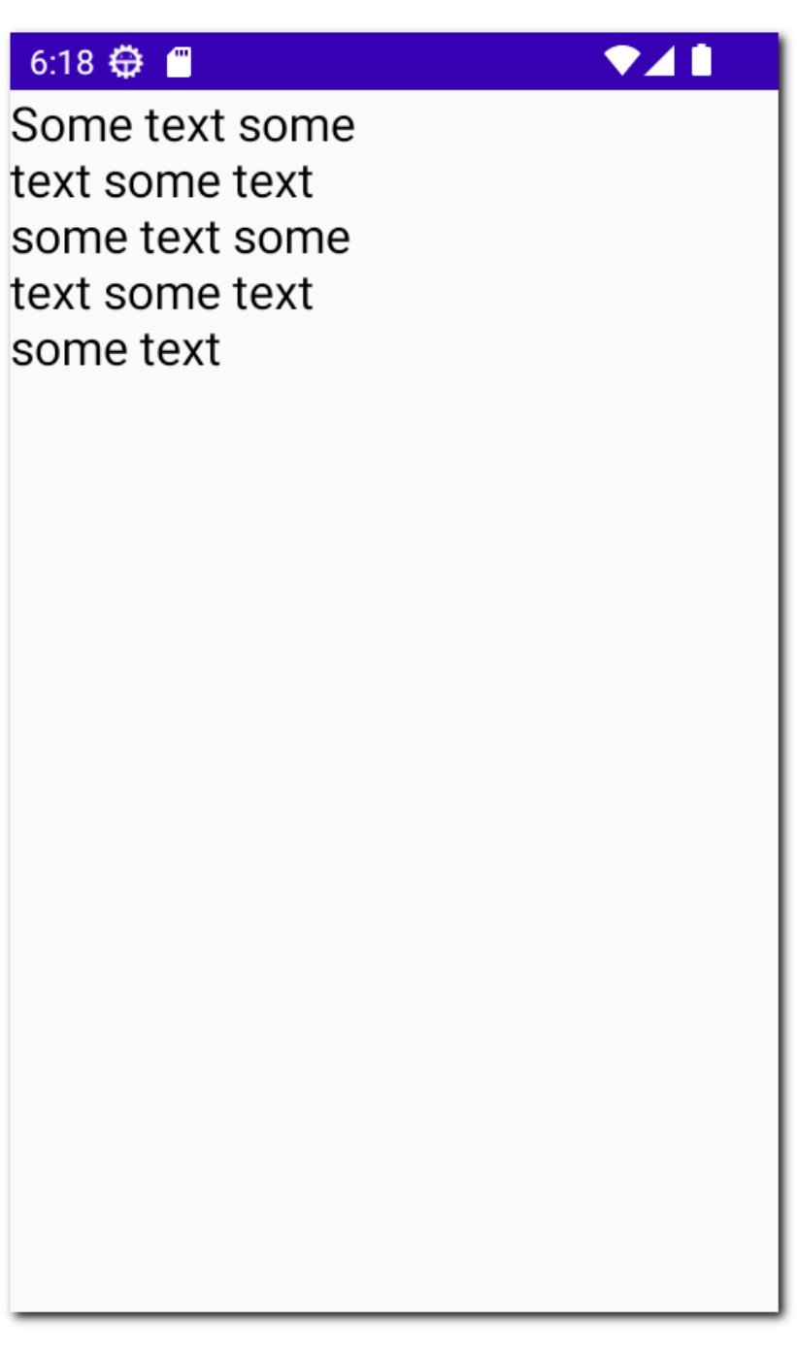
```
Some text
```

textMeasurer дает нам возможность кастомизировать текст.

Например, мы можем добавить стиль и установить ограничения по ширине:

```
val textMeasurer = rememberTextMeasurer()
Canvas(modifier = Modifier.fillMaxSize()) {
    val measuredText = textMeasurer.measure(
        text = AnnotatedString("Some text some text some text some text some text some text some tex
        constraints = Constraints.fixedWidth(size.width.toInt() / 2),
        style = TextStyle(fontSize = 20.sp)
    )
    drawText(measuredText)
}
```

Результат:



```
Some text some
text some text
some text some
text some text
some text
```

Image

Метод drawImage дает возможность вывести на канву картинку

Для Bitmap это выглядит так:

```
val bitmap = ImageBitmap.imageResource(id = R.drawable.sand)
Canvas(modifier = Modifier.fillMaxSize()) {
    drawImage(bitmap)
}
```

Для вектора - чуть сложнее:

```
val vector = ImageVector.vectorResource(id = R.drawable.ic_launcher_foreground)
val painter = rememberVectorPainter(image = vector)
Canvas(modifier = Modifier.fillMaxSize()) {
    with(painter) {
        draw(painter.intrinsicSize, colorFilter = ColorFilter.tint(color = Color.Green))
    }
}
```

Присоединяйтесь к нам в **Telegram**:

- в канале StartAndroid публикуются ссылки на новые статьи с сайта startandroid.ru и интересные материалы с хабра, medium.com и т.п.

- в чатах решаем возникающие вопросы и проблемы по различным темам: Android, Compose, Kotlin, RxJava, Dagger, Тестирование, Performance

- ну и если просто хочется поговорить с коллегами по разработке, то есть чат Флудильня

Назад Вперёд

Комментарии

# **RE: Урок 27. Drawing** — **Boris** 01.11.2023 12:57
Мааааленькая ошибка в первом код сниппете, там размер 200 на 200, а на картинке 200 на 300
Ответить

Обновить список комментариев
RSS лента комментариев этой записи

Добавить комментарий

Тема

Отправить

JComments

© Start Android, 2011-2020. Все права защищены, копирование материалов возможно только с разрешения автора

При возникновении каких-либо вопросов, используйте форму обратной связи

Bootstrap is a front-end framework of Twitter, Inc. Code licensed under MIT License.
Font Awesome font licensed under SIL OFL 1.1.